

Подключение через Entity Framework и Database First

Сначала, для выполнения задания нужно создать новый проект в Visual Studio, а именно проект Windows Forms (Майкрософт). Требуется именно .NET (Core) версия форм, а не .NetFramework, иначе подключение пакетов не сработает.

Для подключения требуются следующие пакеты

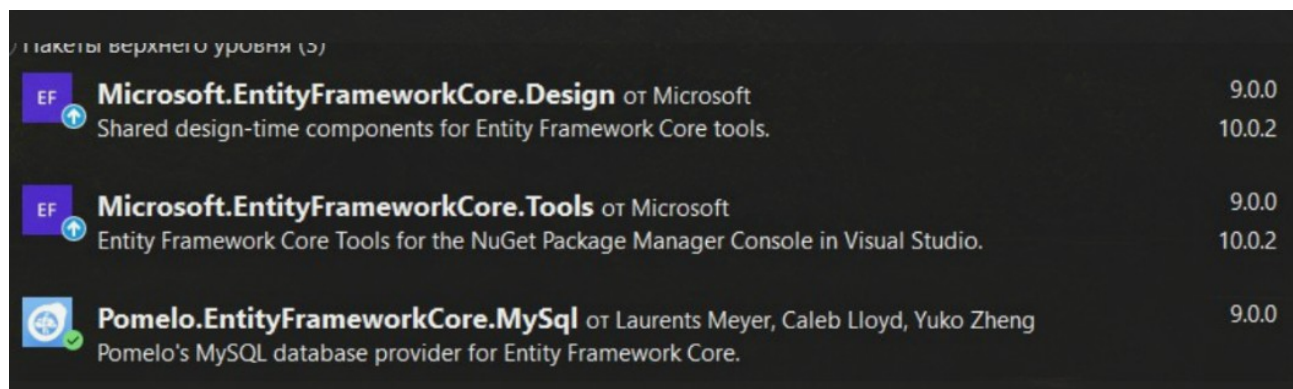
- Microsoft.EntityFrameworkCore.Design 9 версия

- Microsoft.EntityFrameworkCore.Tools 9 версия

Для импорта в проект

Pomelo.EntityFrameworkCore.MySQL

Провайдер



Дальше, требуется найти консоль диспетчера пакетов, если консоли не обнаружено, то следует нажать на вкладку:

Вид > Консоль Диспетчера пакетов.

Далее, введите команду:

Scaffold-DbContext «Ваша строка подклюочения»

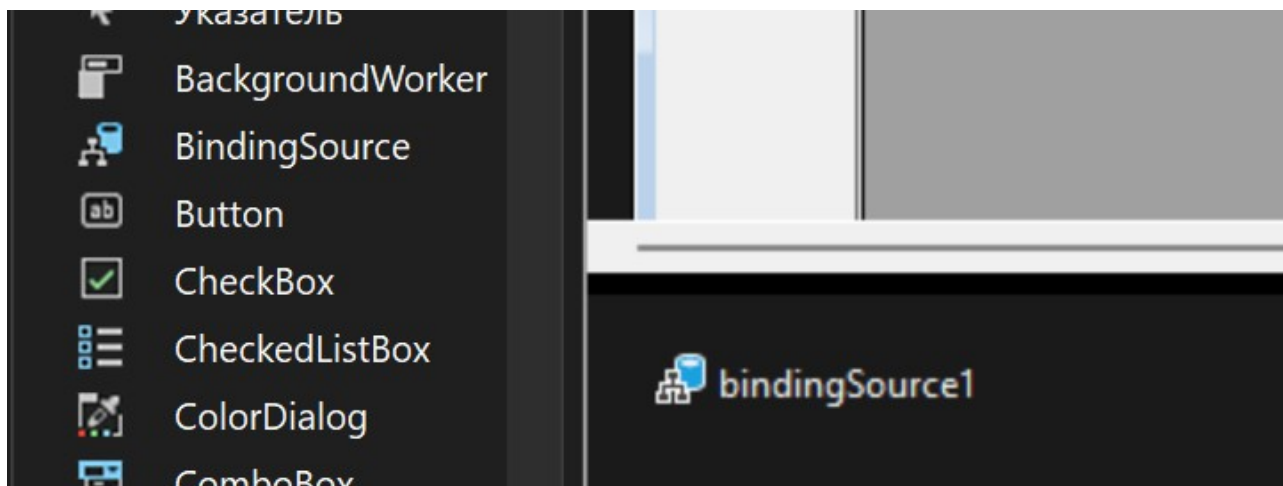
«Pomelo.EntityFrameworkCore.MySQL» 9 версия

Следующая команда: Scaffold-DbContext "server = cfif31.ru; username = username; password=youtpass; database=dbname" "Pomelo.EntityFrameworkCore.MySql" -OutputDir Models -f.

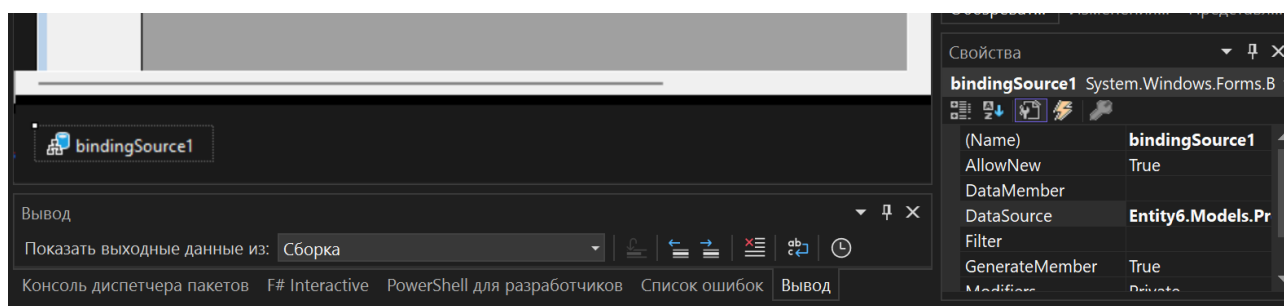
После успешного выполнения команды, выполните сборку проекта **Сборка > Собрать решение.** Это нужно, чтобы Visual Studio смогла получить информацию о типах данных, полученных при сборке.

CRUD через datagridview и Binding Source

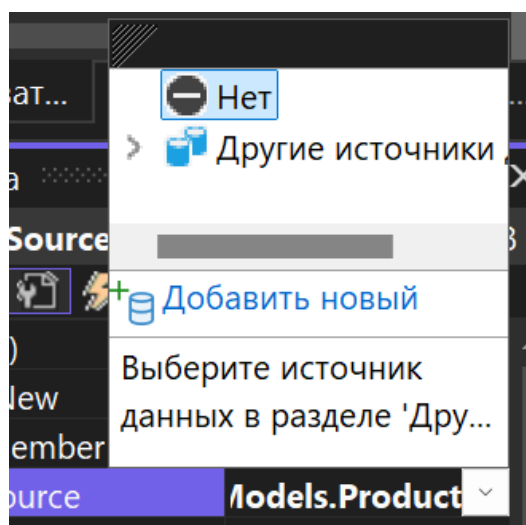
Добавьте на форму компонент BindingSource



Далее, заходите в свойства у BindingSource



Затем требуется нажать на DataSource, а именно на стрелочку справа, чтобы раскрылось специальное меню



Затем выбрать ваши сущности, используемые для вывода

Имя типа	Пространство им...	Имя сборки
Тип источника данных по умолчанию (Entity6.Models)		
☐ Category	Entity6.Models	Entity6, Version=1.0.0.0, Culture...
☐ LazarevTa...	Entity6.Models	Entity6, Version=1.0.0.0, Culture...
☒ Product	Entity6.Models	Entity6, Version=1.0.0.0, Culture...

☒ Показывать только типы, определенные для проекта
 ☒ Скрыть типы, производные от Control

☒ Сначала разместить типы пространств имен проекта
 ☐ Показать только выбранные типы

OK
 Отмена

Требуется нажать ОК. Далее, разместите dataGridView на форме, нажмите стрелочку и выберите созданный источник данных:

DataGridView Задачи

Выбрать источник данных: bindingSource1

Изменить столбцы...
 Добавить столбец...

☒ Включить добавление
 ☒ Включить правку

Запишите созданный на предыдущих шагах класс контекста, состоящий из имени базы данных и слова Context

```

Ссылка: 3
public partial class Form1 : Form
{
    //Класс с генерированным контекстом
    LazarevTaPrepContext context;
    Ссылка: 1
  
```

Затем требуется записать следующую логику в OnLoad().

В инициализации контекста требуется заменить контекст на импортированный.

В загрузке сущностей должен быть указан ваш сгенерированный класс.

Ссылка: 0

```
protected override void OnLoad(EventArgs e)
{
    base.OnLoad(e);

    //инициализация контекста
    context = new LazarevTaPrepContext();

    //Загрузка сущностей из базы
    context.Products.Load();

    //Проверка на создание базы
    context.Database.EnsureCreated();

    //привязка данных
    bindingSource1.DataSource = context.Products.Local.ToBindingList();
}
```

Затем требуется создать кнопку и оформить следующее событие щелчка.

Ссылка: 1

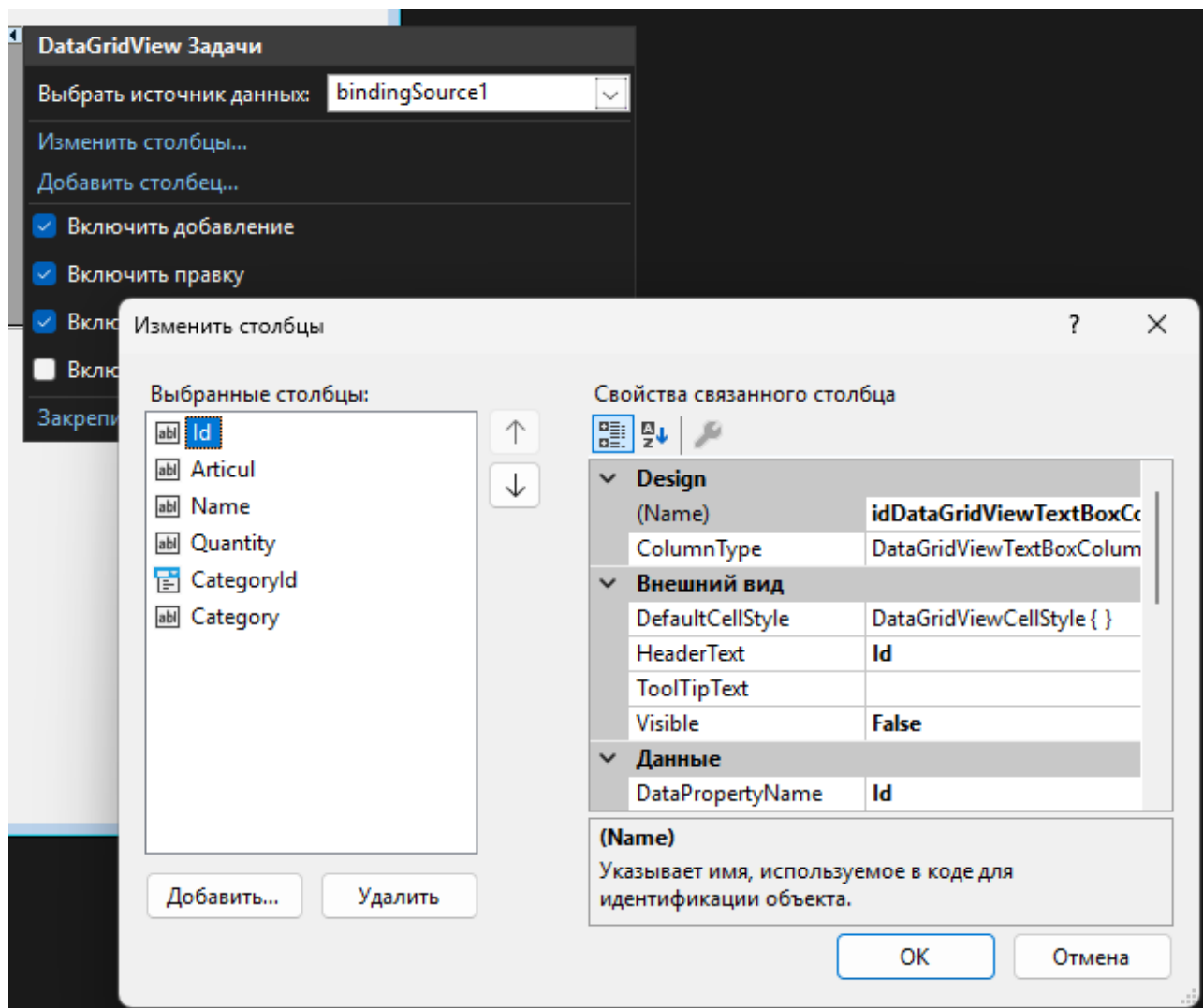
```
private void saveButton_Click(object sender, EventArgs e)
{
    //Сохранение отслеживаемых сущностей в бд
    context.SaveChanges();
    dataGridView1.Refresh();
}
```

После сборки и запуска проекта можно убедиться, что записи в dataGridView можно добавлять, удалять, изменять без написания SQL запросов.

Редактирование внешнего вида колонок

Клиенту не нужно знать об идентификаторах, когда вы работаете с приложением.

Чтобы бы убрать ту или иную колонку, нужно нажать на Изменить столбцы.

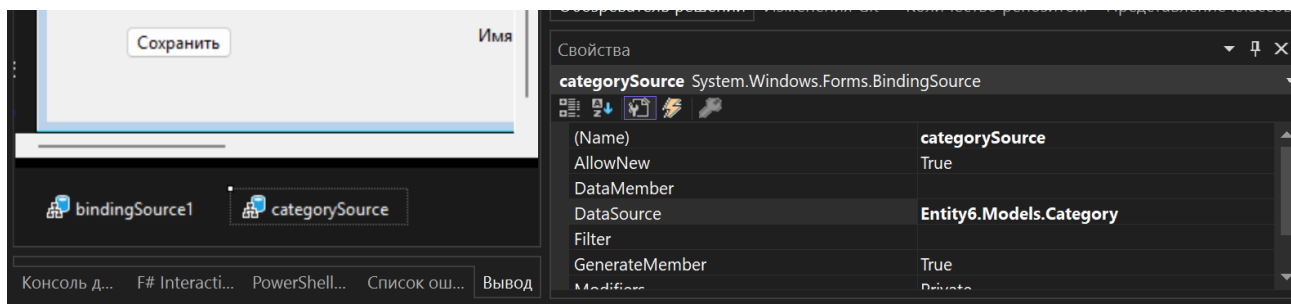


Здесь в категории Внешний вид, интересующими свойствами являются HeaderText и Visible, отвечающими соответственно за название колонки и видимость.

Смежные таблицы

Объединение через колонки Windows Forms

Требуется создать новый источник данных (bindingSource для категорий) и указать ваш класс категорий, который соответствует внешней, связанной сущности.



Следующая логика заполнения

```
{  
    base.OnLoad(e);  
  
    //инициализация контекста  
    context = new LazarevTaPrepContext();  
  
    //Загрузка сущностей из базы  
    context.Products.Include(x => x.Category).Load();  
    context.Categories.Load();  
  
    //Проверка на создание базы  
    context.Database.EnsureCreated();  
  
    //привязка данных  
    bindingSource1.DataSource = context.Products.Local.ToBindingList();  
    categorySource.DataSource = context.Categories.Local.ToBindingList();  
}
```

Далее перейдите в изменение колонок, выберите колонку идентификатором вашей связанной таблицы и измените columnName на указанный в скриншоте и измените колонку для того, чтобы вместо идентификатора показывалось имя товара.

Design	
(Name)	categoryIdDataGridViewTextBoxColumn
ColumnType	DataGridViewComboBoxColumn
Внешний вид	
DefaultCellStyle	DataGridViewCellStyle { }
DisplayStyle	DropDownButton
DisplayStyleForCurrentCellOnly	False
FlatStyle	Standard
HeaderText	Категория
ToolTipText	
Visible	True
Данные	
DataPropertyName	CategoryId
DataSource	categorySource
DisplayMember	Name
Items	(Collection)
ValueMember	Id
Макет	
AutoSizeMode	NotSet
DividerWidth	0

Для просмотра полей в связанной таблице, вы сначала должны добавить несвязанный столбец

Добавить столбец
?
X

☐
Столбец со связанными данными

Столбцы в DataSource

Id
Articul
Name
Quantity
CategoryId
Category

☒
Непривязанный столбец

Имя:
Color1

Тип:
DataGridViewTextBoxColumn

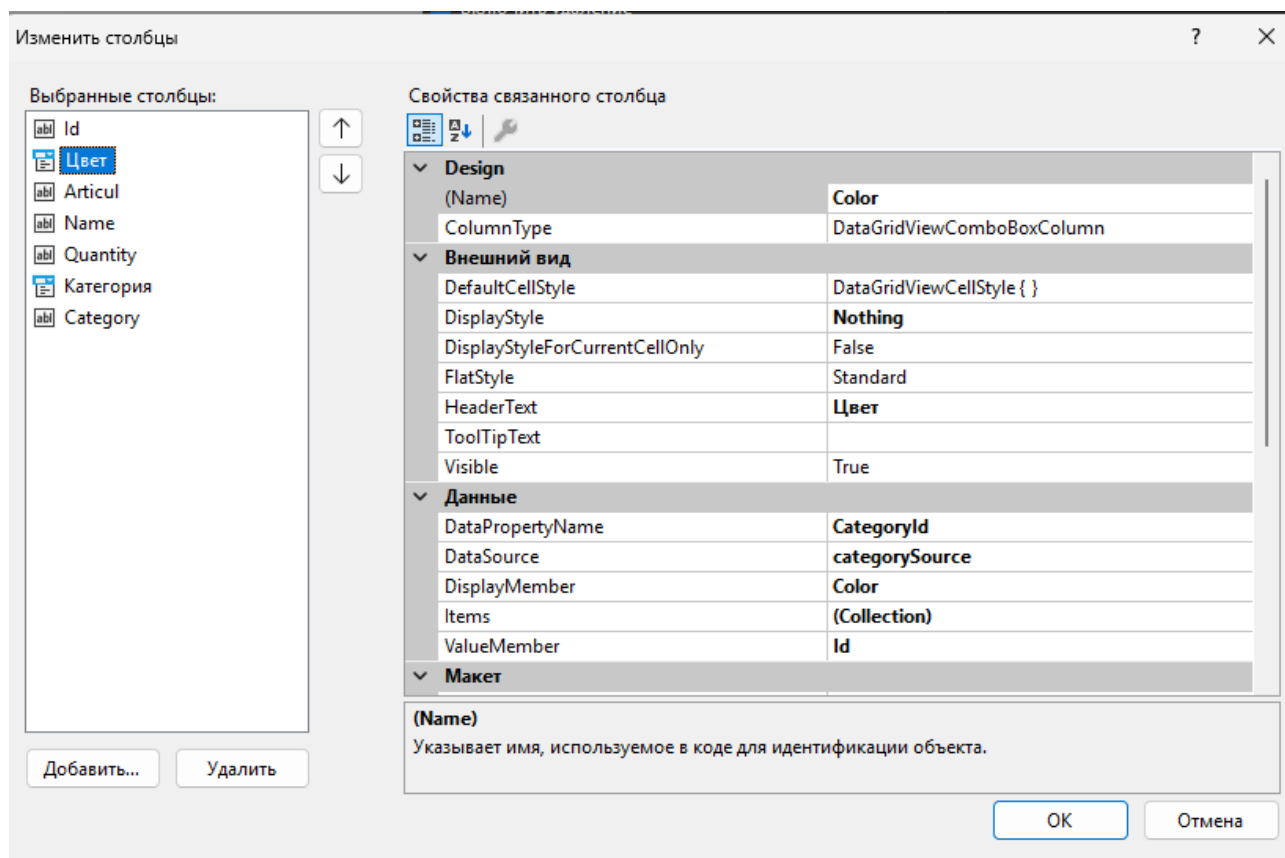
Текст заголовка:
Цвет

☒ Видимый
☐ Только для чтения
☐ Заблокирован

Добавить
Отменить

Далее в ColumnType Нужно подставить DataGridViewComboBoxColumn

В настройках столбцов нужно указать в `DataPropertyName` идентификатор внешней сущности. В `dataSource` указать источник таблицы. В `DisplayMember` нужно подставить интересующую колонку. В `ValueMember` также указать её идентификатор.



Объединение через создание свойств

Требуется повторить шаг с Include в предыдущем пункте. Создайте отдельный cs файл в проекте и добавьте туда partial класс со свойством, которое получает значение из другого вложенного свойства или вычисляет новое значение:

```
namespace WinFormsAppEntity.Models
{
    Ссылка: 13
    public partial class Product
    {
        //Вычисление новой колонки
        [NotMapped]
        Ссылка: 0
        public string NameQuantity => Category.Name + ": " + Name + " " + Quantity.ToString() + " шт." ;

        // Получение вложенной колонки
        [NotMapped]
        Ссылка: 0
        public string CategoryColor => Category.Color;
    }
}
```

После этого требуется пересобрать проект и добавить новую колонку в конструкторе форм

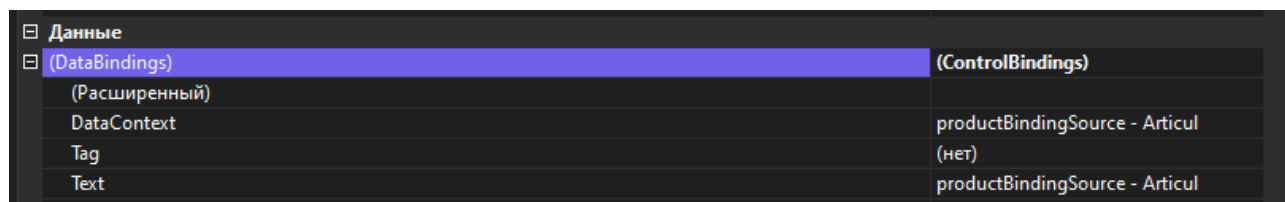
Класс обязательно должен быть `partial` и иметь тот же `namespace`, как и у сгенерированного `ef Product`.

Подробная инструкция, как присоединять таблицы через `Include` (аналог `Join`) в `entity framework` описана в следующей статье

<https://learn.microsoft.com/ru-ru/ef/core/querying/related-data/eager>

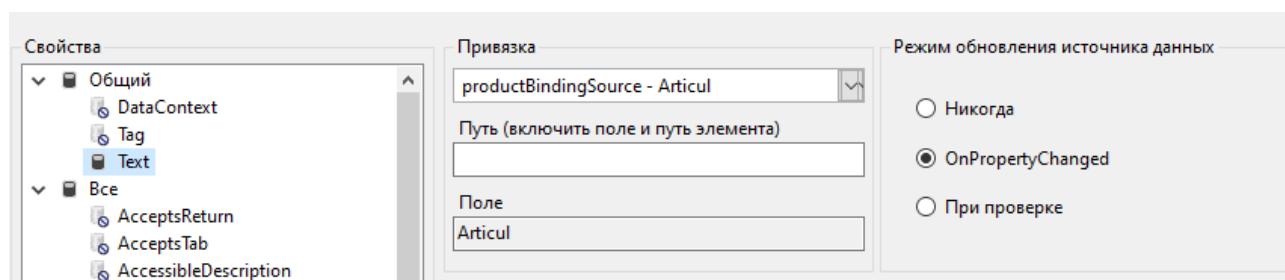
Редактирование записи

Создайте новую форму и добавьте поля. Далее зайдите в один из текстовых и найдите категорию `Данные`, а в неё (`DataBindings`).



(DataBindings)	(ControlBindings)
(Расширенный)	
DataContext	productBindingSource - Articul
Tag	(нет)
Text	productBindingSource - Articul

Затем нужно выбрать меню в поле `расширенный`. Если кликнуть на пустой слот, то появится троеточие, на которое нужно, чтобы открылось следующее меню.

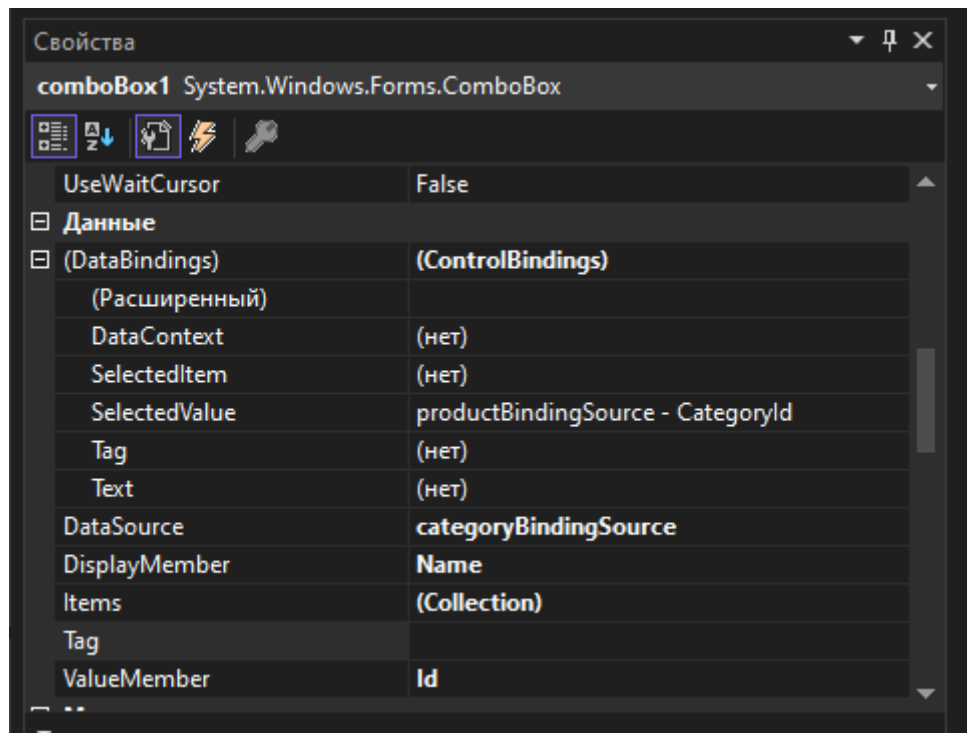


Можно выбрать источник данных для каждого свойства слева, для вывода информации в текстовом поле следует выбрать `Text`

Если вы не создали источник данных для выводимой сущности, то можете выбрать его из существующих классов, а именно из **других источников данных**, где вы можете выбрать подходящий класс.

Режим обновления источника данных следует поставить `OnPropertyChanged`.

Для редактирования связанных сущностей следует воспользоваться отдельными источниками данных. Для настройки привязки требуется создать поле комбобокс и зайти в свойство `Данные`:



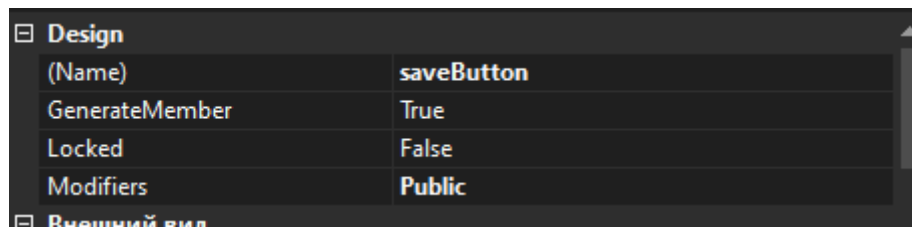
Внутри (DataBindings) поле Selected Value должно соответствовать ключу связанной сущности внутри редактируемой сущности.

DataSource соответствует источнику данных, из которого можно будет выбирать категории.

В DisplayMember можно выбрать атрибут у сущности по которому в комбобоксе будет выводиться текст.

В ValueMember находится первичный ключ сущности, соответствующий внешнему ключу у редактируемой сущности.

Далее требуется создать кнопку с модификатором public, чтобы можно было привязаться к её событиям из другой формы.



Модификатор Public, таким же образом, нужно выставить для всех источников данных в проекте

Затем перейдите на исходную форму, где отображаются все данные и добавьте логику для кнопки редактирования данных.

```
//обработка редактирования
Ссылка: 1
private void editButton_Click(object sender, EventArgs e)
{
    var form = new Form2();
    //Пробрасывание источника данных категорий в форму для редактирования
    form.categoryBindingSource.DataSource = categoryBindingSource.DataSource;
    //Пробрасывание выбранного элемента в источнике данных в форму
    form.productBindingSource.DataSource = bindingSource1.Current;
    //Переиспользование логики сохранения
    form.saveButton.Click += saveButton_Click;
    form.Show();
}

Ссылка: 2
private void saveButton_Click(object sender, EventArgs e)
{
    context.SaveChanges();
    dataGridView1.Refresh();
}
```

Добавление и удаление

Логика добавления и удаления следующая

Ссылка: 1

```
private void add_Click(object sender, EventArgs e)
{
    var form = new Form2();
    form.categoryBindingSource.DataSource = categoryBindingSource.DataSource;
    //одновременно добавляет элемент в источник данных и на форму
    form.productBindingSource.DataSource = bindingSource1.AddNew();
    form.saveButton.Click += saveButton_Click;
    form.Show();
}
```

Ссылка: 1

```
private void delete_Click(object sender, EventArgs e)
{
    //удаляет текущий элемент
    bindingSource1.RemoveCurrent();
    context.SaveChanges();
    dataGridView1.Refresh();
}
```

Валидация сущностей

Валидация происходит с использованием интерфейса `IEEditableObject`, отвечающий за транзакционное добавление сущностей в базу. Эту абстракцию использует `BindingSource` для добавления данных. В ней же можно и проводить валидацию сущностей.

Такой класс может выглядеть следующим образом:

```
using System.ComponentModel;
using System.ComponentModel.DataAnnotations;
namespace WinFormsAppEntity.Models;

Ссылка: 10
public partial class Product : IEEditableObject
{
    LazarevTaPrepContext context;

    Ссылка: 0
    public void BeginEdit()
    {
        context = new();
        context.Update(this);
    }

    Ссылка: 0
    public void CancelEdit()
    {
        var entry = context.Entry(this);
        if (entry.State == Microsoft.EntityFrameworkCore.EntityState.Modified)
        {
            entry.Reload();
        }
    }

    Ссылка: 0
    public void EndEdit()
    {
        //Логика валидации
        if (string.IsNullOrEmpty(Name))
        {
            throw new ValidationException("Имя не должно быть пустым");
        }
        //Сохраняет измененную сущность
        context.SaveChanges();
    }
}
```

В `BeginEdit` инициализируется ваш контекст, который ничего не знает о текущем объекте, поэтому нужен `context.Update`, чтобы EF стал отслеживать его.

В `CancelEdit` присутствует `Reload` для загрузки сущности из баз данных.

`CancelEdit` вызывается напрямую из кода и из `datagridview`, когда строка с проблемой выходит из фокуса

Ссылка: 2

```
private void saveButton_Click(object sender, EventArgs e)
{
    try
    {
        bindingSource1.EndEdit();
    }
    catch (ValidationException error)
    {
        MessageBox.Show(error.Message);
        bindingSource1.CancelEdit();
    }

    dataGridView1.Refresh();
}
```

Ссылка: 1

```
private void add_Click(object sender, EventArgs e)
{
    var form = new Form2();
    form.categoryBindingSource.DataSource = categoryBindingSource.DataSource;
    form.productBindingSource.DataSource = bindingSource1.AddNew();
    form.saveButton.Click += (o, e) =>
    {
        try
        {
            bindingSource1.EndEdit();
        }
        catch (ValidationException error)
        {
            MessageBox.Show(error.Message);
        }
    };
    ;
    form.Show();
}
```

Если при добавлении выйдет исключение, то сущность не будет добавляться как в BindingSource, так и в базу данных

Авторизация

Для следующего примера требуется самостоятельно создать окно авторизации и привязать поля, как в примере с редактированием. Для авторизации может использоваться следующий код:

```
using System.Data;
using WinFormsAppEntity.Models;

namespace WinFormsAppEntity
{
    Ссылка: 3
    public partial class Auth : Form
    {
        User user = new();
        Ссылка: 1
        public Auth()
        {
            InitializeComponent();
        }

        Ссылка: 0
        protected override void OnLoad(EventArgs e)
        {
            base.OnLoad(e);
            //нам не нужно обращаться к textbox
            userBindingSource.DataSource = user;
        }

        Ссылка: 1
        private void enter_Click(object sender, EventArgs e)
        {
            var context = new LazarevTaPrepContext();
            //лямбда выражение для поиска
            var u = context.Users.Where(x => x.Login == user.Login && x.Pwd == user.Pwd).FirstOrDefault();
            if (u != null)
            {
                Hide();
                new Form1().Show();
            }
            else
            {
                MessageBox.Show("Пользователь не найден");
            }
        }
    }
}
```


Создание карточек

Создание карточек включает в себя использование `FlowLayoutPanel` и пользовательского элемента управления. Данный элемент управления используется для автоматического упорядочивания дочерних элементов. Добавьте компонент на экран и проставьте следующие свойства:

Аттрибут	Значение	Комментарий
<code>AutoScroll</code>	<code>true</code>	Автоматическая прокрутка, если элементы не умещаются
<code>FlowDirection</code>	<code>TopDown</code>	Элементы идут сверху вниз
<code>WrapContents</code>	<code>false</code>	Элементы не перекидываются на следующую строку

Для создания пользовательского элемента нажмите правой кнопкой мыши на проект и выберите **Добавить > Пользовательский элемент управления**.

Далее, разместите элементы управления на пользовательском элементе по примеру из параграфа про редактирование и напишите следующую логику в форме.

```
// Заполнение новыми элементами
Ссылка: 2
private void Populate()
{
    flowLayoutPanel1.Controls.Clear(); // Очистка элементов управления
    foreach (var binding in mainBinding) // Получение каждого отдельного элемента связывания
    {
        var control = new CustomElem(); // Создание экземпляра кастомного элемента
        control.productBindingSource.DataSource = binding;
        control.categoryBindingSource.DataSource = catBinding;
        flowLayoutPanel1.Controls.Add(control); // Добавление нового элемента
    }
}
```

Этот код можно вызывать при создании формы и сохранении данных.

```
        mainBinding.DataSource = context.Products.Local.ToBinding();
        catBinding.DataSource = context.Categories.Local.ToBinding();
        Populate();
    }

    //Сохранение данных
    Ссылка: 1
    private void button1_Click(object sender, EventArgs e)
    {
        context.SaveChanges();
        dataGridView1.Refresh();
        Populate();
    }
}
```